

8 YÊU CẦU PHÁT TRIỂN PROJECT

8.1 Giới thiệu

Phần này đưa ra các yêu cầu phát triển thêm cho project, được sắp xếp theo mức độ khó tăng dần. Sinh viên cần hoàn thành các yêu cầu để nâng cao kỹ năng và mở rộng tính năng của hệ thống RAG.

8.2 Các yêu cầu phát triển

8.2.1 Câu hỏi 1: Thêm hỗ trợ file DOCX

Yêu cầu:

- Mở rộng hệ thống để hỗ trợ tải lên và xử lý file DOCX
- Sử dụng thư viện phù hợp (python-docx hoặc DocxLoader từ LangChain)
- Đảm bảo text extraction chính xác

8.2.2 Câu hỏi 2: Lưu trữ lịch sử hội thoại

Yêu cầu:

- Lưu trữ các câu hỏi và câu trả lời trong session
- Hiển thị lịch sử chat trong sidebar
- Cho phép người dùng xem lại các câu hỏi đã hỏi

8.2.3 Câu hỏi 3: Thêm nút xóa lịch sử

Yêu cầu:

- Thêm button "Clear History" để xóa toàn bộ lịch sử chat
- Thêm button "Clear Vector Store" để xóa tài liệu đã upload
- Có confirmation dialog trước khi xóa

8.2.4 Câu hỏi 4: Cải thiện chunk strategy

Yêu cầu:

- Thử nghiệm các giá trị chunk_size khác nhau (500, 1000, 1500, 2000)
- Thử nghiệm chunk_overlap khác nhau (50, 100, 200)
- So sánh và báo cáo kết quả về độ chính xác
- Cho phép người dùng tùy chỉnh chunk parameters

8.2.5 Câu hỏi 5: Thêm citation/source tracking

Yêu cầu:

- Hiển thị nguồn gốc của thông tin (trang số, vị trí trong PDF)
- Cho phép người dùng click để xem context gốc
- Highlight các đoạn văn được sử dụng để trả lời

8.2.6 Câu hỏi 6: Implement Conversational RAG

Yêu cầu:

- Thêm memory để theo dõi ngữ cảnh cuộc hội thoại
- LLM có thể tham chiếu câu hỏi và trả lời trước đó
- Xử lý follow-up questions (câu hỏi tiếp theo)

8.2.7 Câu hỏi 7: Thêm hybrid search

Yêu cầu:

- Kết hợp semantic search (vector) với keyword search (BM25)
- Implement ensemble retriever
- So sánh performance với pure vector search

8.2.8 Câu hỏi 8: Multi-document RAG với metadata filtering

Yêu cầu:

- Hỗ trợ upload nhiều PDF cùng lúc
- Lưu trữ metadata cho mỗi document (tên file, ngày upload, loại)
- Cho phép filter theo metadata khi search
- Hiển thị câu trả lời từ document nào

8.2.9 Câu hỏi 9: Implement Re-ranking với Cross-Encoder

Yêu cầu:

- Thêm bước re-ranking sau retrieval
- Sử dụng cross-encoder model để đánh giá lại relevance
- So sánh với bi-encoder (current approach)
- Tối ưu hóa latency

8.2.10 Câu hỏi 10: Advanced RAG với Self-RAG

Yêu cầu:

- Implement Self-RAG: LLM tự đánh giá câu trả lời
- Query rewriting: Tự động cải thiện câu hỏi
- Multi-hop reasoning
- Confidence scoring

8.3 Hướng dẫn đánh giá

Mỗi câu hỏi được đánh giá theo:

- **Functionality (40%)**: Hoạt động đúng
- **Code Quality (20%)**: Clean code, comments
- **Documentation (20%)**: README, docs
- **Testing (10%)**: Test cases
- **UI/UX (10%)**: User-friendly

Điểm thưởng:

- Performance optimization: +10%
- Error handling: +5%
- Creative solutions: +5%
- Comprehensive tests: +10%